Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Συστήματα Πολυμέσων

Advanced Audio Coding (AAC)

Τζίνα Θεοδώρα
10715
tzinatheod@ece.auth.gr

Χειμερινό Εξάμηνο 2025-2026

Περιεχόμενα

1 Εισαγωγή **3**

2 Επίπεδο 1 (Level 1) **4**
2.1 Sequence Segmentation Control (SSC) 4
2.2 Filterbank 6
2.3 Encoder / Decoder 1 8

3 Επίπεδο 2 (Level 2) **10**
3.1 Temporal Noise Shaping (TNS) 10
3.2 Encoder / Decoder 2 12

4 Επίπεδο 3 (Level 3) **14**
4.1 Psychoacoustic Model 14
4.2 Quantizer 18
4.3 Κωδικοποίηση Huffman 21
4.4 Encoder / Decoder 3 21

1 Εισαγωγή

Η παρούσα εργασία περιγράφει την υλοποίηση ενός απλοποιημένου συστήματος κωδικοποιητή και αποκωδικοποιητή ήχου, το οποίο βασίζεται στο ευρέως διαδεδομένο πρότυπο **Advanced Audio Coding (AAC)**. Η συγκεκριμένη αρχιτεκτονική κωδικοποίησης ανήκει στην κατηγορία της συμπίεσης κυματομορφής (waveform compression). Βασικός της στόχος είναι η αναπαράσταση του σήματος με τέτοιο τρόπο, ώστε η αποκωδικοποιημένη εκδοχή του να προσεγγίζει ακουστικά το αρχικό ηχητικό σήμα όσο το δυνατόν περισσότερο.

Για τις ανάγκες της υλοποίησης, η επεξεργασία εφαρμόζεται σε στερεοφωνικό σήμα ήχου με συχνότητα δειγματοληψίας 48000 samples/sec. Το αρχικό σήμα χωρίζεται σε επικαλυπτόμενα (κατά 50%) τμήματα (frames) μήκους 2048 δειγμάτων. Στη συνέχεια, κάθε frame υφίσταται αυτόνομη και σειριακή επεξεργασία.

Σύμφωνα με την αρχιτεκτονική του προτύπου, όπως αυτή αποτυπώνεται στα διαγράμματα του συστήματος, το αρχικό σήμα εισόδου ακολουθεί μια συγκεκριμένη ροή επεξεργασίας. Αρχικά, το σήμα τροφοδοτείται παράλληλα σε τρεις βαθμίδες: στο **Psychoacoustic Model**, το οποίο υπολογίζει τα κατώφλια ακουστότητας , στο **Sequence Segmentation Control (SSC)**, το οποίο αποφασίζει τον τύπο του εκάστοτε frame , και στο **Filter Bank**. Το Filter Bank δέχεται την πληροφορία ελέγχου από το SSC και μετασχηματίζει τα δείγματα από το πεδίο του χρόνου στο πεδίο της συχνότητας (MDCT). Στη συνέχεια, οι συντελεστές περνούν από τη βαθμίδα **Temporal Noise Shaping (TNS)** για τον έλεγχο της κατανομής του σφάλματος κβαντισμού και έπειτα από τη βαθμίδα Mid/Side Stereo (M/S), η οποία μετατρέπει το δικαναλικό σήμα (στην παρούσα απλοποιημένη υλοποίηση παραλείπεται). Έπειτα, τα δεδομένα καταλήγουν στον **Quantizer** (Κβαντιστή), ο οποίος, καθοδηγούμενος από τα κατώφλια του ψυχοακουστικού μοντέλου (και θεωρητικά από το Bit reservoir, το οποίο επίσης παραλείπεται), κβαντίζει τους συντελεστές. Τέλος, τα παραγόμενα σύμβολα συμπιέζονται περαιτέρω στη βαθμίδα **Huffman** και πολυπλέκονται (Bitstream MUX) για την παραγωγή του τελικού bitstream. Η διαδικασία της αποκωδικοποίησης ακολουθεί την ακριβώς αντίστροφη και σειριακή πορεία, περνώντας διαδοχικά από τις βαθμίδες Bitstream DEMUX, iHuffman, iQuantizer, iM/S, iTNS και, τελικά, iFilter Bank για την πλήρη ανακατασκευή του ακουστικού σήματος.

2 Επίπεδο 1 (Level 1)

Στο πρώτο επίπεδο της εργασίας, υλοποιείται ο βασικός κορμός του κωδικοποιητή και αποκωδικοποιητή ήχου κατά το πρότυπο AAC. Η υλοποίηση σε αυτό το στάδιο εστιάζει στη διαχείριση του ηχητικού σήματος (δικαναλικός ήχος) και στη μετατροπή του από το πεδίο του χρόνου στο πεδίο της συχνότητας. Για να επιτευχθεί αυτό, αναπτύχθηκαν οι θεμελιώδεις βαθμίδες ελέγχου κατάτμησης της ακολουθίας (Sequence Segmentation Control - SSC), καθώς και οι συστοιχίες φίλτρων (Filterbanks). Τέλος, το επίπεδο αυτό ολοκληρώνεται με τη δημιουργία των κεντρικών συναρτήσεων κωδικοποίησης και αποκωδικοποίησης (Encoder / Decoder). Στις επόμενες υποενότητες αναλύεται τόσο θεωρητικά όσο και προγραμματιστικά η λειτουργία της κάθε βαθμίδας.

2.1 Sequence Segmentation Control (SSC)

Η βαθμίδα Sequence Segmentation Control (SSC) είναι υπεύθυνη για την επιλογή του κατάλληλου τύπου παραθύρου (frame) για την κωδικοποίηση του σήματος. Υπάρχουν τέσσερις τύποι frames:

- **ONLY_LONG_SEQUENCE (OLS):** Βασικός τύπος που χρησιμοποιείται όταν τα φασματικά χαρακτηριστικά του σήματος παραμένουν σχετικά αμετάβλητα.
- **EIGHT_SHORT_SEQUENCE (ESH):** Βασικός τύπος που επιλέγεται όταν υπάρχουν έντονες μεταβολές στο σήμα (transients) κατά τη διάρκεια του frame.
- **LONG_START_SEQUENCE (LSS):** Συμπληρωματικός/μεταβατικός τύπος που παρεμβάλλεται όταν μεταβαίνουμε από OLS σε ESH.
- **LONG_STOP_SEQUENCE (LPS):** Συμπληρωματικός/μεταβατικός τύπος που παρεμβάλλεται κατά τη μετάβαση από ESH σε OLS.

Η απόφαση για τον τύπο του τρέχοντος frame i εξαρτάται από τον τύπο του προηγούμενου frame $i-1$ και από την ανάλυση του επόμενου frame $i+1$. Για τον έλεγχο του επόμενου frame, τα δείγματά του φιλτράρονται πρώτα με ένα υψιπερατό φίλτρο:

$$H(z) = \frac{0.7548 - 0.7548z^{-1}}{1 - 0.5095z^{-1}}$$

Στη συνέχεια, σε 8 επιμέρους περιοχές (μήκους 128 δειγμάτων) υπολογίζεται η ενέργεια s_l^2 και η τιμή attack:

$$ds_l^2 = \frac{s_l^2}{\frac{1}{l}\sum_{m=0}^{l-1} s_m^2}$$

Αν εντοπιστεί $s_l^2 > 10^{-3}$ και $ds_l^2 > 10$ σε έστω και ένα τμήμα (εκτός του πρώτου), το επόμενο frame θεωρείται ESH (δηλαδή εντοπίστηκε απότομη αλλαγή/attack). Η διαδικασία αυτή εκτελείται ανεξάρτητα για κάθε κανάλι του στερεοφωνικού σήματος και η τελική κοινή απόφαση λαμβάνεται βάσει ενός προκαθορισμένου πίνακα λογικής.

Υλοποίηση και Δομή Κώδικα:
Στην υλοποίησή μας, η βαθμίδα αναπαρίσταται από τη συνάρτηση `SSC(frame_T, next_frame_T, prev_frame_type)`. Η διαδικασία που ακολουθήθηκε στον κώδικα αποτυπώνει πιστά τα θεωρητικά βήματα:

1. **Φιλτράρισμα:** Ορίζονται οι συντελεστές `b` και `a` του υψιπερατού φίλτρου και εφαρμόζονται στο επόμενο frame (`next_frame_T`) κάνοντας χρήση της συνάρτησης `scipy.signal.lfilter`.

   Listing 1: Φιλτράρισμα επόμενου frame

   ```
   # High-Pass Filter coefficients
   b = [0.7548, -0.7548]
   a = [1.0, -0.5095]
   filtered = lfilter(b, a, next_frame_T[:, ch])
   ```

2. **Απομόνωση Κεντρικής Περιοχής:** Από το φιλτραρισμένο σήμα αποκόπτονται τα πρώτα 448 δείγματα, ώστε να διατηρηθεί η κεντρική περιοχή των 1152 δειγμάτων.

   Listing 2: Απομόνωση κεντρικής περιοχής

   ```
   # Extract central region (448:1600)
   center_region = filtered[448:448+1152]
   ```

3. **Υπολογισμός Ενεργειών και Attacks:** Για καθένα από τα δύο κανάλια ανεξάρτητα, το σήμα χωρίζεται σε 8 τμήματα των 128 δειγμάτων. Υπολογίζεται το άθροισμα των τετραγώνων (ενέργεια `s2`) και κατόπιν υπολογίζεται ο λόγος `ds2`.

   Listing 3: Υπολογισμός ενεργειών και τιμών attack

   ```
   # Compute energies
   segment = center_region[l*128:(l+1)*128]
   s2[l] = np.sum(segment ** 2)

   # Compute attack values
   mean_prev = np.mean(s2[:l])
   ds2[l] = s2[l] / mean_prev if mean_prev > 1e-10 else 0.0
   ```

4. **Λήψη Απόφασης ανά Κανάλι:** Ελέγχονται τα κατώφλια για να τεθεί η boolean μεταβλητή `has_attack`. Με βάση αυτήν και τον τύπο του προηγούμενου frame (`prev_frame_type`), καθορίζεται ο προτεινόμενος τύπος (π.χ., αν είχαμε LSS, τότε πάμε υποχρεωτικά σε ESH, ενώ αν είχαμε OLS και βρέθηκε attack, πάμε σε LSS).

   Listing 4: Ανίχνευση attack και λήψη απόφασης ανά κανάλι

   ```
   # Check for transient
   if s2[l] > 1e-3 and ds2[l] > 10:
       has_attack = True
   ```

```
# Decide frame type for THIS channel
if prev_frame_type == "LSS":
    ch_type = "ESH"
...
```

5. **Τελική Επιλογή:** Οι επιμέρους αποφάσεις του αριστερού και δεξιού καναλιού συνδυάζονται χρησιμοποιώντας ένα λεξικό (`decision_table`), το οποίο αποτελεί την προγραμματιστική αποτύπωση του "Πίνακα 1" της εκφώνησης.

Η υλοποίηση της συγκεκριμένης βαθμίδας βρίσκεται στο αρχείο `aac_ssc.py`.

2.2 Filterbank

Η πληροφορία του τύπου κάθε frame διαβιβάζεται στη βαθμίδα Filterbank, η οποία χρησιμοποιεί τον μετασχηματισμό Modified Discrete Cosine Transform textbf(MDCT) για να μειώσει τη συσχέτιση των δειγμάτων, μεταβαίνοντας ταυτόχρονα από το πεδίο του χρόνου στο πεδίο της συχνότητας. Ο μετασχηματισμός MDCT αποτελεί παραλλαγή του DCT η οποία λειτουργεί σε επικαλυπτόμενα παράθυρα ώστε να αποφεύγει τα προβλήματα που εμφανίζονται στα όρια των παραθύρων. Έτσι, για N δείγματα παραθύρου $s_{i,n}$, ο MDCT οδηγεί σε $N/2$ συντελεστές $X_{i,k}$, σύμφωνα με τη σχέση:

$$X_{i,k} = 2\sum_{n=0}^{N-1} s_{i,n} \cos\left(\frac{2\pi}{N}(n+n_0)\left(k+\frac{1}{2}\right)\right), \quad 0 \le k < \frac{N}{2}$$

όπου $n_0 = (N/2+1)/2$. Αντίστοιχα, ο αντίστροφος μετασχηματισμός (IMDCT) που χρησιμοποιείται στην αποκωδικοποίηση δίνεται από τη σχέση:

$$s_{i,n} = \frac{2}{N}\sum_{k=0}^{\frac{N}{2}-1} X_{i,k} \cos\left(\frac{2\pi}{N}(n+n_0)\left(k+\frac{1}{2}\right)\right), \quad 0 \le n < N$$

.

Πριν την εφαρμογή του MDCT (και μετά τον IMDCT), τα δείγματα πολλαπλασιάζονται στο πεδίο του χρόνου με κατάλληλα παράθυρα, είτε τύπου Kaiser-Bessel-Derived (KBD) είτε ημιτονοειδή (SIN). Η διαδικασία παραθύρωσης διαφοροποιείται ανάλογα με τον τύπο του frame:

- Για τα OLS frames χρησιμοποιείται συμμετρικό παράθυρο $N = 2048$.
- Για τα LSS και LPS frames χρησιμοποιούνται μη συμμετρικά παράθυρα $N = 2048$ (με τμήματα από άσους και μηδενικά για ομαλή μετάβαση).
- Για τα ESH frames, απομονώνονται τα 1152 κεντρικά δείγματα και αγνοούνται τα 448 δείγματα εκατέρωθεν. Το σήμα χωρίζεται σε 8 επικαλυπτόμενα κατά 50% subframes μήκους $N = 256$, καθένα από τα οποία παραθυρώνεται και μετασχηματίζεται αυτόνομα.

Υλοποίηση και Δομή Κώδικα:
Στην υλοποίησή μας, η διαδικασία χωρίζεται λογικά στις συναρτήσεις `filter_bank(frame_T, frame_type, win_type)` και `i_filter_bank(frame_F, frame_type, win_type)`, με τη βοήθεια εξωτερικών συναρτήσεων υπολογισμού (`mdct`, `i_mdct`, `kbd_window`, `sin_window`). Η πορεία που ακολουθήθηκε είναι η εξής:

1. **Παραγωγή Συναρτήσεων Παραθύρου:** Αρχικά δημιουργούνται τα μεγάλα (2048) και μικρά (256) παράθυρα ανάλογα με τον επιλεγμένο τύπο (KBD ή SIN).

2. **Σύνθεση Ασύμμετρων Παραθύρων** (π.χ. LSS): Ανάλογα με τον τύπο του frame, κατασκευάζεται το τελικό παράθυρο. Για παράδειγμα, στα LSS frames δημιουργείται το ασύμμετρο παράθυρο ενώνοντας τμήματα.

Listing 5: Κατασκευή μη συμμετρικού παραθύρου για LONG_START_SEQUENCE

```
elif frame_type == "LSS":
    # LONG_START_SEQUENCE: transition to short
    window = np.concatenate([
        w_long[:1024],     # Left half long (0:1024)
        np.ones(448),      # Flat top (1024:1472)
        w_short[128:],     # Right half short (1472:1600)
        np.zeros(448)      # Zero padding (1600:2048)
    ])
```

3. **Διαχείριση Short Frames (ESH) και Εφαρμογή MDCT:** Στα short frames, το κεντρικό τμήμα τεμαχίζεται σε 8 subframes με 50% επικάλυψη. Σε κάθε subframe εφαρμόζεται παραθύρωση και MDCT.

Listing 6: Επεξεργασία των 8 subframes για EIGHT_SHORT_SEQUENCE

```
# Process 8 overlapping subframes (50% overlap)
for i in range(8):
    start_idx = i * 128
    end_idx = start_idx + 256
    subframe = center_samples[start_idx:end_idx]

    # Apply short window and MDCT
    windowed = subframe * w_short
    frame_F[:, i, ch] = mdct(windowed, 256)
```

4. **Αντίστροφη Διαδικασία (IMDCT και Overlap-Add):** Κατά την αποκωδικοποίηση, εφαρμόζεται ο IMDCT και εκ νέου παραθύρωση. Ιδιαίτερα για τα ESH, τα ανακατασκευασμένα subframes προστίθενται αθροιστικά (overlap-add) στις σωστές χρονικές θέσεις.

Listing 7: Διαδικασία Overlap-Add για τα subframes στην i_filter_bank

```
# Overlap-add at correct position in frame
```

```
start_idx = 448 + i * 128
end_idx = start_idx + 256
frame_T[start_idx:end_idx, ch] += windowed
```

Η υλοποίηση των κεντρικών βαθμίδων βρίσκεται στο αρχείο `aac_filterbank.py`, ενώ οι βοηθητικές μαθηματικές συναρτήσεις (παράθυρα και MDCT/IMDCT) βρίσκονται στο `aac_utils.py`. Για την επιτάχυνση των υπολογισμών, έχει ενσωματωθεί τεχνική caching (αποθήκευση στην μνήμη) του πίνακα συνημιτόνων στο MDCT/IMDCT.

2.3 Encoder / Decoder 1

Η συνολική διαδικασία του 1ου επιπέδου ενορχηστρώνεται από τις βαθμίδες του κωδικοποιητή και του αποκωδικοποιητή. Αρχικά, το προς κωδικοποίηση σήμα ήχου διαβάζεται από ένα αρχείο μορφής wav, με την υπόθεση ότι πρόκειται για δικαναλικό (στερεοφωνικό) ήχο με συχνότητα δειγματοληψίας 48kHz. Το αρχικό σήμα χωρίζεται σε επικαλυπτόμενα κατά 50% τμήματα (frames) μήκους 2048 δειγμάτων.
Ο κωδικοποιητής αναλαμβάνει να τροφοδοτήσει διαδοχικά κάθε frame στη βαθμίδα Sequence Segmentation Control και έπειτα στη βαθμίδα Filterbank. Το τελικό αποτέλεσμα του κωδικοποιητή είναι μία λίστα `aac_seq_1` που περιέχει τόσα στοιχεία όσα και τα frames. Κάθε στοιχείο είναι ένα λεξικό (dictionary) που αποθηκεύει τον τύπο του frame, τον τύπο του παραθύρου και τους συντελεστές MDCT για το αριστερό και το δεξί κανάλι.
Αντίστροφα, ο αποκωδικοποιητής διαβάζει τη δομή `aac_seq_1`, εφαρμόζει την αντίστροφη βαθμίδα `i_filter_bank` για να ανακατασκευάσει το σήμα στο πεδίο του χρόνου και, μέσω της τεχνικής overlap-add (λόγω της επικάλυψης 50%), συνθέτει την τελική αποκωδικοποιημένη ακολουθία δειγμάτων, η οποία εγγράφεται σε ένα νέο αρχείο wav.

Υλοποίηση και Δομή Κώδικα:
Στην υλοποίησή μας, η δόμηση του 1ου επιπέδου βρίσκεται στο αρχείο `aac_codec_1.py`, το οποίο περιλαμβάνει τις συναρτήσεις `aac_coder_1(filename_in)`, `i_aac_coder_1(aac_seq_1, filename_out)` και `demo_aac_1(filename_in, filename_out)`.

1. **Προετοιμασία Σήματος:** Το σήμα διαβάζεται μέσω της βιβλιοθήκης `soundfile`. Εφαρμόζεται zero-padding στην αρχή και στο τέλος για να διασφαλιστεί η σωστή επικάλυψη (overlap) των παραθύρων MDCT και η σωστή λειτουργία του SSC lookahead.

Listing 8: Ανάγνωση και Padding του ηχητικού σήματος

```
# Read audio file
audio, fs = sf.read(filename_in)

# Pad with 1024 zeros at beginning and end for proper MDCT overlap
audio = np.pad(audio, ((hop_size, hop_size), (0, 0)), mode='constant')
```

2. **Κωδικοποίηση:** Για κάθε frame, εξάγονται το τρέχον και το επόμενο τμήμα. Καλείται η SSC και έπειτα η `filter_bank`. Το αποτέλεσμα αποθηκεύεται στη λίστα `aac_seq_1`.

Listing 9: Βρόχος κωδικοποίησης ανά frame

```
# Store encoded frame (example for OLS/LSS/LPS)
aac_seq_1.append({
        'frame_type': frame_type,
        'win_type': win_type,
        'chl': {'frame_F': frame_F[:, 0]},
        'chr': {'frame_F': frame_F[:, 1]}
})
```

3. **Ανακατασκευή Σήματος (Αποκωδικοποίηση):** Ο αποκωδικοποιητής διατρέχει τη λίστα `aac_seq_1`, καλεί την `i_filter_bank` και αθροίζει (overlap-add) τα ανακατασκευασμένα frames.

4. **Υπολογισμός SNR:** Στη συνάρτηση `demo_aac_1`, καλούνται οι παραπάνω διαδικασίες και συγκρίνεται το αρχικό με το αποκωδικοποιημένο σήμα για τον υπολογισμό του Σηματοθορυβικού Λόγου (SNR).

Listing 10: Υπολογισμός συνολικού SNR

```
# Calculate SNR
signal_power = np.sum(original ** 2)
noise_power = np.sum((original - decoded) ** 2)
SNR = 10 * np.log10(signal_power / noise_power)
```

Το σύνολο των συναρτήσεων αυτών βρίσκεται στο αρχείο `aac_codec_1.py`, ενώ ένα επιπλέον test script (`aac_test_1.py`) υλοποιήθηκε για την εύκολη εκτέλεση της επίδειξης:

Listing 11: Αποτελέσματα Επιπέδου 1

```
Audio Quality:
------------------------------------------------------------
SNR: 253.99 dB

Quality: Excellent
```

Η εξαιρετικά υψηλή τιμή του SNR (253.99 dB) υποδεικνύει ότι η διαφορά μεταξύ του αρχικού και του αποκωδικοποιημένου σήματος είναι πρακτικά μηδαμινή. Δεδομένου ότι στο 1ο Επίπεδο δεν έχει ενσωματωθεί ακόμα καμία βαθμίδα απωλεστικής συμπίεσης (όπως ο κβαντιστής) και δεν απορρίπτεται πληροφορία βάσει του ψυχοακουστικού μοντέλου, η διαδικασία περιορίζεται αποκλειστικά στην παραθύρωση και τον μετασχηματισμό MDCT.
Το ελάχιστο υπολειπόμενο σφάλμα, το οποίο αποτρέπει το SNR από το να τείνει στο άπειρο, δεν προέρχεται από αλγοριθμική απώλεια πληροφορίας, αλλά οφείλεται αποκλειστικά στα σφάλματα στρογγυλοποίησης λόγω της πεπερασμένης ακρίβειας κινητής υποδιαστολής (floating-point errors).

3 Επίπεδο 2 (Level 2)

Στο δεύτερο επίπεδο της εργασίας, το σύστημα κωδικοποίησης επεκτείνεται με την ενσωμάτωση της βαθμίδας Temporal Noise Shaping (TNS). Η βαθμίδα αυτή αποτελεί ένα κρίσιμο εργαλείο για τον έλεγχο και τη διαμόρφωση του σφάλματος κβαντισμού, το οποίο αποτελεί συχνό πρόβλημα σε ηχητικά σήματα που παρουσιάζουν απότομες χρονικές μεταβολές (transients). Στις επόμενες παραγράφους αναλύεται θεωρητικά και πρακτικά ο τρόπος με τον οποίο το TNS εφαρμόζει τεχνικές γραμμικής πρόβλεψης (Linear Prediction) στους συντελεστές MDCT στο πεδίο της συχνότητας, προκειμένου να προσαρμόσει την κατανομή του θορύβου στο σχήμα του αρχικού σήματος στο πεδίο του χρόνου.

3.1 Temporal Noise Shaping (TNS)

Η αποτελεσματική εφαρμογή του ψυχοακουστικού μοντέλου στον AAC προϋποθέτει ότι το παρατηρούμενο σήμα είναι στάσιμο κατά τη διάρκεια του παραθύρου και, συνεπώς, το σφάλμα κβαντισμού των συντελεστών MDCT κατανέμεται ομοιόμορφα στο πεδίο του χρόνου. Ωστόσο, σε "μεταβατικά" παράθυρα όπου εμφανίζονται έντονοι ήχοι (π.χ. παλμοί), αυτή η παραδοχή καταρρέει, οδηγώντας σε αντιληπτές παραμορφώσεις γύρω από τον παλμό. Το Temporal Noise Shaping (TNS) επιλύει αυτό το πρόβλημα εκμεταλλευόμενο τη δυαδικότητα χρόνου/συχνότητας: εφαρμόζει γραμμική πρόβλεψη (Linear Prediction) στους συντελεστές MDCT στο πεδίο της συχνότητας, προσαρμόζοντας το σφάλμα κβαντισμού στο σχήμα του σήματος στο πεδίο του χρόνου. Η διαδικασία που ακολουθείται είναι η εξής:

1. **Κανονικοποίηση:** Οι συντελεστές MDCT, $X(k)$, κανονικοποιούνται ως προς την ενέργεια $P(j)$ της μπάντας j του ψυχοακουστικού μοντέλου στην οποία ανήκουν, μέσω του συντελεστή $S_w(k) = \sqrt{P(j)}$. Επειδή οι συντελεστές αυτοί αλλάζουν απότομα, εξομαλύνονται πριν την εφαρμογή τους.

2. **Γραμμική Πρόβλεψη:** Υπολογίζονται οι συντελεστές $a_1, \ldots, a_p$ (σταθερής τάξης $p = 4$) που ελαχιστοποιούν το τετραγωνικό σφάλμα πρόβλεψης. Η εύρεση των βέλτιστων συντελεστών απαιτεί τη λύση των κανονικών εξισώσεων $Ra = r$, όπου R ο πίνακας και r το διάνυσμα αυτοσυσχέτισης.

3. **Κβαντισμός:** Οι παραπάνω συντελεστές κβαντίζονται με 4 bits χρησιμοποιώντας ομοιόμορφο συμμετρικό κβαντιστή με βήμα 0.1.

4. **Φιλτράρισμα:** Εφαρμόζεται το FIR φίλτρο:

$$H_{TNS}(z) = 1 - a_1 z^{-1} - a_2 z^{-2} - \cdots - a_p z^{-p}$$

   στους αρχικούς συντελεστές $X(k)$. Κατά τον υπολογισμό, ελέγχεται προληπτικά η ευστάθεια του αντίστροφου φίλτρου H_{TNS}^{-1} ελέγχοντας αν οι ρίζες του πολυωνύμου (πόλοι) βρίσκονται εντός του μοναδιαίου δίσκου ($|z| < 1$).

Υλοποίηση και Δομή Κώδικα:
Η υλοποίηση της βαθμίδας γίνεται μέσω της συνάρτησης `tns(frame_F_in, frame_type)`, η οποία επιστρέφει τους τροποποιημένους συντελεστές και τους κβαντισμένους συντελεστές TNS. Αντίστοιχα, η συνάρτηση `i_tns(frame_F_in, frame_type, tns_coeffs)` πραγματοποιεί την αντίστροφη διαδικασία. Η λογική της υλοποίησης περιλαμβάνει:

1. **Υπολογισμός Ενεργειών και Εξομάλυνση:** Διατρέχονται οι μπάντες, υπολογίζεται η ενέργεια $P(j)$ και εφαρμόζεται η κανονικοποίηση και το smoothing.

   Listing 12: Υπολογισμός ενέργειας και εξομάλυνση συντελεστών

   ```
   # Calculate energy per band
   P[j] = np.sum(X[w_low:w_high + 1] ** 2)

   # Create Sw array for normalization
   Sw[w_low:w_high + 1] = np.sqrt(P[j]) if P[j] > 0 else 1.0

   # Smooth Sw to avoid sharp transitions
   for k in range(N-2, -1, -1):
       Sw[k] = (Sw[k] + Sw[k+1]) / 2.0
   ...
   ```

2. **Αυτοσυσχέτιση και Επίλυση Εξισώσεων:** Για τη λύση του $Ra = r$, υπολογίζεται η ακολουθία αυτοσυσχέτισης r και λύνεται το γραμμικό σύστημα.

   Listing 13: Λύση των κανονικών εξισώσεων

   ```
   # Compute LP coefficients
   try:
       a = np.linalg.solve(R, r[1:p+1])
   except np.linalg.LinAlgError:
       return X.copy(), np.zeros(4)
   ```

3. **Κβαντισμός και Έλεγχος Ευστάθειας:** Οι συντελεστές a κβαντίζονται και ελέγχονται με χρήση της `numpy.polynomial.polynomial.Polynomial.roots()`.

   Listing 14: Έλεγχος ευστάθειας πόλων

   ```
   # Check stability
   coeffs = np.concatenate([-a_quantized[::-1], [1]])
   roots = np.polynomial.polynomial.Polynomial(coeffs).roots()
   if not np.all(np.abs(roots) < 1.0):
       return X.copy(), np.zeros(4)
   ```

4. **Εφαρμογή Φίλτρου:** Τέλος, το `frame_F_in` φιλτράρεται χρησιμοποιώντας τους κβαντισμένους συντελεστές.

Η υλοποίηση της συγκεκριμένης βαθμίδας βρίσκεται στο αρχείο `aac_tns.py`.

3.2 Encoder / Decoder 2

Στο δεύτερο επίπεδο, η συνολική ροή της κωδικοποίησης και αποκωδικοποίησης αναβαθμίζεται με την προσθήκη της βαθμίδας Temporal Noise Shaping (TNS). Η βαθμίδα TNS παρεμβάλλεται αμέσως μετά τον υπολογισμό των συντελεστών MDCT από το Filterbank. Ο κωδικοποιητής `aac_coder_2` διατρέχει τα frames, αποφασίζει τον τύπο τους (SSC), τα μετασχηματίζει (Filterbank) και στη συνέχεια εφαρμόζει το TNS. Το παραγόμενο αποτέλεσμα είναι η λίστα `aac_seq_2`, η οποία πλέον, εκτός από τον τύπο του frame και του παραθύρου, αποθηκεύει για το κάθε κανάλι τόσο τους τροποποιημένους συντελεστές MDCT όσο και τους κβαντισμένους συντελεστές γραμμικής πρόβλεψης του TNS.
Κατά την αποκωδικοποίηση, η συνάρτηση `i_aac_coder_2` ακολουθεί την αντίστροφη πορεία. Πριν την εφαρμογή του αντίστροφου μετασχηματισμού (IMDCT), καλείται η βαθμίδα `i_tns`. Χρησιμοποιώντας τους κβαντισμένους συντελεστές TNS, το αντίστροφο φίλτρο εφαρμόζεται στους συντελεστές MDCT για να αναιρέσει τη διαμόρφωση, και έπειτα τα δεδομένα προωθούνται στο inverse Filterbank για την ανακατασκευή της ακολουθίας δειγμάτων στο πεδίο του χρόνου.

Υλοποίηση και Δομή Κώδικα:
Στην υλοποίησή μας, οι διαδικασίες αυτές ενσωματώνονται στο αρχείο `aac_codec_2.py`. Τα βασικά βήματα αναλύονται ως εξής:

1. **Ενσωμάτωση TNS στον Κωδικοποιητή:** Αφού εξαχθούν οι συντελεστές MDCT από το Filterbank, καλείται η συνάρτηση `tns` ξεχωριστά για κάθε κανάλι.

   Listing 15: Κλήση της βαθμίδας TNS

   ```
   # Apply TNS to each channel
   frame_F_left_tns, tns_coeffs_left = tns(frame_F[:, 0], frame_type)
   frame_F_right_tns, tns_coeffs_right = tns(frame_F[:, 1], frame_type)

   # Store encoded frame
   aac_seq_2.append({
       'frame_type': frame_type,
       'win_type': win_type,
       'chl': {'frame_F': frame_F_left_tns, 'tns_coeffs': tns_coeffs_left},
       'chr': {'frame_F': frame_F_right_tns, 'tns_coeffs': tns_coeffs_right}
   })
   ```

2. **Αντίστροφο TNS / Αποκωδικοποίηση:** Αντίστοιχα, ο αποκωδικοποιητής εξάγει τους συντελεστές από το dictionary και εφαρμόζει την `i_tns` πριν προχωρήσει στην `i_filter_bank`.

3. **Υπολογισμός SNR:** Η διαδικασία ολοκληρώνεται με τη συνάρτηση `demo_aac_2`, η οποία εκτελεί την πλήρη αλυσίδα κωδικοποίησης-αποκωδικοποίησης, υπολογίζοντας το τελικό SNR.

Το σύνολο των συναρτήσεων αυτών βρίσκεται στο αρχείο `aac_codec_2.py`, ενώ ένα επιπλέον test script (`aac_test_2.py`) υλοποιήθηκε για την εύκολη εκτέλεση της επίδειξης:

Listing 16: Αποτελέσματα Επιπέδου 2

```
Audio Quality:
------------------------------------------------------------
SNR: 253.99 dB

Quality: Excellent
```

Όπως παρατηρούμε, η τιμή του SNR στο 2ο Επίπεδο παραμένει πρακτικά αμετάβλητη (στα 253.99 dB) σε σύγκριση με το 1ο Επίπεδο. Το αποτέλεσμα αυτό είναι απολύτως αναμενόμενο και επιβεβαιώνει την ορθή θεωρητική και πρακτική λειτουργία της βαθμίδας Temporal Noise Shaping (TNS).

Παρότι στο στάδιο αυτό εισάγεται για πρώτη φορά η έννοια του κβαντισμού (οι συντελεστές γραμμικής πρόβλεψης του φίλτρου κβαντίζονται με 4 bits), ο κβαντισμός αυτός δεν εφαρμόζεται πάνω στην πληροφορία του ίδιου του ηχητικού σήματος. Εφόσον τόσο ο κωδικοποιητής κατά την εφαρμογή του FIR φίλτρου (H_{TNS}), όσο και ο αποκωδικοποιητής κατά την εφαρμογή του αντίστροφου φίλτρου (H_{TNS}^{-1}), χρησιμοποιούν ακριβώς τους ίδιους κβαντισμένους συντελεστές, η διαδικασία του φιλτραρίσματος παραμένει μαθηματικά πλήρως αντιστρέψιμη.

Δεδομένου ότι η κύρια βαθμίδα κβάντισης (Quantizer) των συντελεστών MDCT εξακολουθεί να απουσιάζει, το σύστημα συνεχίζει να λειτουργεί χωρίς απώλειες πληροφορίας (lossless). Το ελάχιστο υπολειπόμενο σφάλμα προκύπτει και πάλι αποκλειστικά από την πεπερασμένη ακρίβεια κινητής υποδιαστολής (floating-point precision) κατά την εκτέλεση των πράξεων του φίλτρου.

4 Επίπεδο 3 (Level 3)

Στο τρίτο και τελευταίο επίπεδο της εργασίας, το σύστημα κωδικοποίησης/αποκωδικοποίησης ολοκληρώνεται, προσεγγίζοντας την πλήρη λειτουργικότητα της απλοποιημένης εκδοχής του προτύπου AAC. Σε αυτό το στάδιο εισάγονται οι κρισιμότερες βαθμίδες για την επίτευξη του βασικού στόχου της συμπίεσης. Αρχικά, το Ψυχοακουστικό Μοντέλο (Psychoacoustic Model) αναλύει το σήμα για να υπολογίσει τα κατώφλια ακουστότητας και τον λόγο Σήματος προς Μάσκα (Signal-to-Mask Ratio - SMR), εκμεταλλευόμενο τις ιδιότητες της ανθρώπινης ακοής. Στη συνέχεια, ο Κβαντιστής (Quantizer) χρησιμοποιεί αυτή την πληροφορία για να κβαντίσει τους συντελεστές MDCT, επιτρέποντας την εισαγωγή σφάλματος (θορύβου κβαντισμού) αποκλειστικά κάτω από το κατώφλι ακουστότητας. Τέλος, η βαθμίδα Κωδικοποίησης Huffman (Huffman Coding) αναλαμβάνει την εντροπική συμπίεση των παραγόμενων συμβόλων κβαντισμού χωρίς περαιτέρω απώλειες. Η συνολική αρχιτεκτονική του 3ου επιπέδου ενορχηστρώνει αυτές τις διαδικασίες, επιτρέποντάς μας να μετρήσουμε όχι μόνο την ποιότητα του ανακατασκευασμένου σήματος (SNR), αλλά και το τελικό bitrate και τον λόγο συμπίεσης του συστήματος.

4.1 Psychoacoustic Model

Για τη μείωση των απαιτούμενων bit κατά την κβάντιση, το πρότυπο AAC κάνει χρήση του ανθρώπινου ψυχοακουστικού μοντέλου. Σκοπός του είναι να υπολογίσει τα κατώφλια ακουστότητας για κάθε συχνότητα, καθορίζοντας το επιτρεπόμενο περιθώριο σφάλματος (θορύβου κβαντισμού). Οποιοσδήποτε ήχος βρίσκεται κάτω από αυτό το κατώφλι δεν κωδικοποιείται. Η διαδικασία περιλαμβάνει τα εξής στάδια:

1. **Spreading Function:** Υπολογίζεται η συνάρτηση κατανομής μάσκας (spreading function) για όλους τους συνδυασμούς συχνοτήτων, η οποία περιγράφει πώς μια συχνότητα επικαλύπτει (mask) τις γειτονικές της. Η συνάρτηση υλοποιήθηκε απευθείας στο βήμα 6 όπου γίνεται και η χρήση της.

2. **Παραθύρωση και FFT:** Κάθε frame ή sub-frame πολλαπλασιάζεται με ένα παράθυρο Hann στο πεδίο του χρόνου:

$$s_w(n) = s(n)\left(0.5 - 0.5\cos\left(\frac{\pi(n+0.5)}{N}\right)\right)$$

   Έπειτα, υπολογίζεται ο Μετασχηματισμός Fourier (FFT) για να εξαχθεί το μέτρο $r(w)$ και η φάση $f(w)$ σε κάθε διακριτή συχνότητα w.

3. **Πρόβλεψη Μεγέθους και Φάσης:** Υπολογίζονται οι προβλέψεις $r_{pred}(w)$ και $f_{pred}(w)$ για το τρέχον παράθυρο, κάνοντας χρήση των τιμών από τα δύο ακριβώς προηγούμενα παράθυρα.

4. **Μέτρο Προβλεψιμότητας (Predictability):** Βάσει της παραπάνω πρόβλεψης, υπολογίζεται το μέτρο $c(w)$:

$$c(w) = \frac{\sqrt{(r(w)\cos(f(w)) - r_{pred}(w)\cos(f_{pred}(w)))^2 + (r(w)\sin(f(w)) - r_{pred}(w)\sin(f_{pred}(w)))^2}}{r(w) + |r_{pred}(w)|}$$

5. **Ενέργεια και Προβλεψιμότητα:** Αθροίζονται οι τιμές για όλες τις συχνότητες εντός κάθε μπάντας b του ψυχοακουστικού μοντέλου (βάσει των στηλών w_{low} και w_{high} των πινάκων του προτύπου):

$$e(b) = \sum_{w=w_{low}(b)}^{w_{high}(b)} r(w)^2, \quad c(b) = \sum_{w=w_{low}(b)}^{w_{high}(b)} c(w)r(w)^2$$

6. **Εφαρμογή της Spreading Function:** Συνδυάζονται η ενέργεια και η προβλεψιμότητα με τη spreading function για να βρεθούν τα $ecb(b)$ και $ct(b)$, τα οποία έπειτα κανονικοποιούνται, παράγοντας τα $cb(b)$ και $en(b)$.

7. **Δείκτης Τονικότητας:** Υπολογίζεται ο δείκτης τονικότητας $tb(b)$, ο οποίος λαμβάνει τιμές στο $(0, 1)$:

$$tb(b) = -0.299 - 0.43\ln(cb(b))$$

8. **Απαιτούμενο SNR:** Χρησιμοποιώντας τις σταθερές Noise Masking Tone ($NMT =$ 6dB) και Tone Masking Noise ($TMN =$ 18dB), υπολογίζεται το απαιτούμενο SNR:

$$SNR(b) = tb(b)TMN(b) + (1 - tb(b))NMT(b)$$

9. **Μετατροπή σε Λόγο Ενέργειας:** Το SNR μετατρέπεται από dB σε γραμμική κλίμακα.

10. **Κατώφλι Ενέργειας:** Υπολογίζεται το κατώφλι ενέργειας ως $nb(b) = en(b)bc(b)$.

11. **Επίπεδο Θορύβου:** Για κάθε μπάντα, το τελικό επίπεδο θορύβου κβαντισμού $npart(b)$ ορίζεται ως το μέγιστο μεταξύ του κατωφλίου λόγω masking (nb) και του κατωφλίου σε απόλυτη ησυχία ($\hat{q}_{thr}$):

$$npart(b) = \max\{nb(b), \hat{q}_{thr}(b)\}$$

12. **Signal-to-Mask Ratio (SMR):**

$$SMR(b) = \frac{e(b)}{npart(b)}$$

13. **Κατώφλια Κωδικοποιητή $T(b)$:** Τα κατώφλια ακουστότητας υλοποιήθηκαν απευθείας στην βαθμίδα του Κβαντιστή.

Υλοποίηση και Δομή Κώδικα:
Στην υλοποίησή μας, σημαντική επιλογή αποτελεί ο διαχωρισμός της λογικής σε δύο συναρτήσεις: την κύρια `psycho(frame_T, frame_type, frame_T_prev_1, frame_T_prev_2)` και τη βοηθητική `_process_frame`. Ο λόγος για αυτόν τον διαχωρισμό είναι η αποφυγή επανάληψης κώδικα για τους διαφορετικούς τύπους παραθύρων. Η μαθηματική ακολουθία για τον υπολογισμό του SMR (παραθύρωση, FFT, spreading function, υπολογισμός κατωφλίων) είναι ακριβώς η ίδια ανεξάρτητα από το αν το σήμα είναι ένα πλήρες frame 2048 δειγμάτων (OLS, LSS, LPS) ή ένα μικρό subframe 256 δειγμάτων (ESH).
Έτσι, η `psycho` αναλαμβάνει τη διαχείριση των frames και subframes, ενώ η `_process_frame` λειτουργεί ως ένας γενικός υπολογιστικός πυρήνας που εφαρμόζεται πάνω στα δεδομένα που της τροφοδοτούνται. Τα βασικά στάδια της υλοποίησης αναλύονται ως εξής:

1. **Προετοιμασία και Διαχείριση frames:** Αρχικά, η συνάρτηση `psycho` διαχειρίζεται τα edge cases των πρώτων frames του ηχητικού σήματος, όπου δεν υπάρχουν προηγούμενα frames (θέτοντάς τα σε πίνακες μηδενικών). Στη συνέχεια, διαχωρίζει τη ροή ανάλογα με τον τύπο:

   - Για τα **Long Frames (OLS/LSS/LPS)**, απλώς φορτώνει τον πίνακα ορίων B.2.1.9.a και καλεί τη βοηθητική συνάρτηση.
   - Για τα **Short Frames (ESH)**, φορτώνει τον πίνακα B.2.1.9.b και αναλαμβάνει την πολύπλοκη παρακολούθηση της ιστορίας των subframes. Επειδή κάθε frame περιέχει 8 subframes (δείκτες 0 έως 7), για τον υπολογισμό της προβλεψιμότητας του subframe i απαιτούνται τα subframes $i-1$ και $i-2$. Για $i=0$ και $i=1$, η αναζήτηση επιστρέφει στο αμέσως προηγούμενο frame, ανακτώντας τα τελευταία του subframes (6 και 7).

   Listing 17: Λογική εύρεσης προηγούμενων subframes στα ESH frames

```
for i in range(8):
    # Determine the two preceding subframes
    ...
    elif i == 1:
        # Need subframe 0 of current and subframe 7 of previous
        sub_prev_1 = curr_subs[0]
        sub_prev_2 = prev1_subs[7]
    ...
```

2. **Κύριος Μαθηματικός Υπολογισμός:** Μέσα στη βοηθητική συνάρτηση `_process_frame` εκτελούνται σειριακά όλα τα βασικά μαθηματικά βήματα του προτύπου. Αρχικά εφαρμόζεται παράθυρο Hann και μετασχηματισμός FFT στο τρέχον, το προηγούμενο και το προ-προηγούμενο παράθυρο για την εξαγωγή του μέτρου (r) και της φάσης (f). Έπειτα, υπολογίζονται οι προβλέψεις (`r_pred`, `f_pred`) και το μέτρο προβλεψιμότητας (`c_pred`) για κάθε διακριτή συχνότητα. Στη συνέχεια, οι συχνότητες ομαδοποιούνται στις καθορισμένες μπάντες του ψυχοακουστικού μοντέλου (βάσει των ορίων `w_low` και `w_high`), υπολογίζοντας την ενέργεια `e[b]` και

τη σταθμισμένη προβλεψιμότητα c[b] για την καθεμία. Αμέσως μετά, εφαρμόζεται η συνάρτηση διασποράς (Spreading Function) για να συνυπολογιστεί η αλληλεπίδραση των γειτονικών συχνοτήτων, και τα μεγέθη κανονικοποιούνται.

Listing 18: Υπολογισμός ενέργειας (e) και προβλεψιμότητας (c) ανά μπάντα

```
# Critical band energy and predictability
for b in range(num_bands):
    w_low = int(bands[b, 1]) # Lower FFT bin
    w_high = min(int(bands[b, 2]), N//2) # Upper FFT bin

    if w_low < N//2:
        e[b] = np.sum(r[w_low:w_high + 1]**2)
        c[b] = np.sum(c_pred[w_low:w_high + 1] * r[w_low:w_high + 1]**2)
```

3. **Δείκτης Τονικότητας και Κατώφλι Μάσκας:** Αφού ολοκληρωθεί η εφαρμογή της spreading function, το επόμενο στάδιο υπολογίζεται ο δείκτης τονικότητας (tb) για κάθε μπάντα, ο οποίος δείχνει αν η μπάντα προσομοιάζει περισσότερο σε τόνο ή σε θόρυβο. Βάσει αυτού και των σταθερών τιμών NMT (6dB) και TMN (18dB), υπολογίζεται το απαιτούμενο SNR. Το SNR μετατρέπεται από λογαριθμική σε γραμμική κλίμακα (λόγος ενέργειας bc), προκειμένου να πολλαπλασιαστεί με την κανονικοποιημένη ενέργεια (en) και να εξαχθεί το τελικό κατώφλι μάσκας ενέργειας (nb).

Listing 19: Υπολογισμός Δείκτη Τονικότητας και Κατωφλίου Μάσκας

```
# Compute tonality index t[b]
tb = np.clip(-0.299 - 0.43 * np.log(cb + 1e-10), 0, 1)

# Compute SNR
SNR = tb * TMN + (1 - tb) * NMT
```

4. **Εξαγωγή Τελικού SMR:** Στο τελευταίο στάδιο, το κατώφλι μάσκας (nb) συγκρίνεται με το απόλυτο κατώφλι ακουστότητας σε ησυχία (qthr). Το μέγιστο από τα δύο αποτελεί το τελικό επιτρεπόμενο επίπεδο θορύβου κβαντισμού (npart). Τέλος, ο λόγος της ενέργειας του σήματος (e) προς αυτόν τον θόρυβο μάς δίνει το επιθυμητό Signal-to-Mask Ratio (SMR).

Listing 20: Υπολογισμός τελικού επιπέδου θορύβου και SMR

```
# Compute absolute hearing threshold (qthr)
qthr = np.finfo(float).eps * (N / 2) * 10**(qsthr / 10.0)

# Apply maximum between masking threshold and absolute hearing threshold
npart = np.maximum(nb, qthr)
```

Το αποτέλεσμα είναι η επιστροφή του πίνακα SMR, ο οποίος θα καθοδηγήσει τα βήματα κβάντισης στην επόμενη βαθμίδα του συστήματος, διασφαλίζοντας ότι το σφάλμα κβαντισμού

παραμένει μη αντιληπτό από το ανθρώπινο αυτί. Η διαδικασία του ψυχοακουστικού μοντέλου αναπτύσσεται στο αρχείο `aac_psycho.py`.

4.2 Quantizer

Ο κβαντιστής αποτελεί τη βαθμίδα στην οποία επιτυγχάνεται η ουσιαστική συμπίεση του σήματος, καθώς είναι το μόνο σημείο του συστήματος που εισάγει απώλειες πληροφορίας (lossy compression). Ο σκοπός του είναι να μειώσει την ακρίβεια των συντελεστών MDCT, εισάγοντας ένα ελεγχόμενο σφάλμα (θόρυβο κβαντισμού), το οποίο όμως, χάρη στα κατώφλια $T(b)$ του ψυχοακουστικού μοντέλου, θα παραμείνει μη αντιληπτό από το ανθρώπινο αυτί. Η βαθμίδα αυτή παρουσιάζει αρκετή πολυπλοκότητα και λειτουργεί ως εξής:

1. **Ομαδοποίηση σε Scalefactor Bands:** Για να περιοριστεί ο όγκος της πληροφορίας που πρέπει να μεταδοθεί στον αποκωδικοποιητή, δεν ορίζεται διαφορετικό βήμα κβάντισης για κάθε επιμέρους συντελεστή συχνότητας. Αντιθέτως, οι συντελεστές ομαδοποιούνται σε ζώνες (οι οποίες στην απλοποιημένη υλοποίησή μας ταυτίζονται με τις μπάντες του ψυχοακουστικού μοντέλου). Σε κάθε ζώνη b, εφαρμόζεται ένας κοινός παράγοντας κλίμακας, ο λεγόμενος *scalefactor gain* $\alpha(b)$, ο οποίος ρυθμίζει την "ποιότητα" του κβαντισμού για όλους τους συντελεστές της συγκεκριμένης ζώνης.

2. **Μη-Ομοιόμορφη Κβάντιση:** Η διαδικασία κβάντισης (στον κωδικοποιητή) και αποκβάντισης (στον αποκωδικοποιητή) εκφράζεται αντίστοιχα από τις σχέσεις:

$$S(k) = \text{sgn}(X(k)) \cdot \text{int}\left[\left(|X(k)| \cdot 2^{-0.25\alpha(b)}\right)^{0.75} + \text{MagicNumber}\right]$$

$$\hat{X}(k) = \text{sgn}(S(k)) \cdot |S(k)|^{\frac{4}{3}} \cdot 2^{0.25\alpha(b)}$$

   Η σταθερά MagicNumber $= 0.4054$ έχει επιλεγεί πειραματικά για τη βελτιστοποίηση της ακέραιας στρογγυλοποίησης. Όσο μεγαλύτερη είναι η τιμή του $\alpha(b)$, τόσο περισσότερο συμπιέζεται το σήμα, αυξάνοντας όμως το σφάλμα.

3. **Επαναληπτικός Βρόχος Εύρεσης $\alpha(b)$:** Ο κεντρικός πυρήνας της βαθμίδας είναι η εύρεση του βέλτιστου $\alpha(b)$ για κάθε μπάντα, η οποία γίνεται επαναληπτικά:

   (a) **Αρχική Εκτίμηση Υψηλής Ποιότητας:** Ο αλγόριθμος ξεκινάει υπολογίζοντας μια αρχική τιμή $\hat{\alpha}(b)$, η οποία προσφέρει τη μέγιστη δυνατή ποιότητα, αλλά καταναλώνει υπερβολικά πολλά bits:

$$\hat{\alpha}(b) = \frac{16}{3}\log_2\left(\frac{\max_k(|X(k)|)^{0.75}}{MQ}\right)$$

   (b) **Μέτρηση Σφάλματος:** Σε κάθε επανάληψη, οι συντελεστές κβαντίζονται και αμέσως αποκβαντίζονται. Το πραγματικό σφάλμα που εισάγεται υπολογίζεται

ως η ενέργεια της διαφοράς μεταξύ αρχικών και αποκβαντισμένων συντελεστών:

$$P_e(b) = \sum_{k=w_{low}(b)}^{w_{high}(b)} (X(k) - \hat{X}(k))^2$$

(c) **Σταδιακή Υποβάθμιση:** Αν το σφάλμα $P_e(b)$ είναι μικρότερο από το κατώφλι ακουστότητας $T(b)$, σημαίνει ότι βρισκόμαστε κάτω από το όριο της ανθρώπινης ακοής και "σπαταλάμε" bits. Συνεπώς, η τιμή του $\alpha(b)$ αυξάνεται κατά 1, και η διαδικασία επαναλαμβάνεται.

(d) **Τερματισμός:** Ο βρόχος της συγκεκριμένης μπάντας σταματά όταν το σφάλμα $P_e(b)$ υπερβεί το κατώφλι $T(b)$ (δηλαδή φτάσαμε στο μέγιστο όριο συμπίεσης χωρίς ακουστή παραμόρφωση). Επιπλέον, εισάγεται ένας αυστηρός κανόνας ελέγχου που αποτρέπει απότομες μεταβολές που θα μείωναν την απόδοση της μετέπειτα κωδικοποίησης Huffman: $\max_b |\alpha(b+1) - \alpha(b)| \leq 60$.

4. **Διαφορική Κωδικοποίηση:** Τέλος, αντί να ενσωματωθούν στο bitstream οι απόλυτες τιμές των $\alpha(b)$, το πρότυπο εκμεταλλεύεται τη συσχέτιση γειτονικών μπαντών. Αποθηκεύεται η απόλυτη τιμή της πρώτης μπάντας ως *Global Gain* ($G = \alpha(0)$) και για τις υπόλοιπες μπάντες υπολογίζεται η διαφορά τους από την προηγούμενη: $sfc(b) = \alpha(b) - \alpha(b-1)$. Αυτές οι διαφορές (DPCM) επικεντρώνονται γύρω από το μηδέν και είναι ιδανικές για συμπίεση μέσω Huffman.

Υλοποίηση και Δομή Κώδικα:
Όπως και στο ψυχοακουστικό μοντέλο, η αρχιτεκτονική που υιοθετήθηκε διαχωρίζει τη διαδικασία στις κύριες συναρτήσεις `aac_quantizer(frame_F, frame_type, SMR)`, `i_aac_quantizer(S, sfc, G, frame_type)` και σε βοηθητικές συναρτήσεις (`_quantize_subframe`, `_quantize`, `_dequantize`, `_dequantize_subframe`). Τα βασικά στάδια και η ροή εκτέλεσης αναλύονται ως εξής:

1. **Διαχείριση Τύπων Frame:** Η κύρια συνάρτηση `aac_quantizer`παραλαμβάνει τους συντελεστές MDCT και φορτώνει τους πίνακες των ορίων ανάλογα με τον τύπο του frame. Για τα μεγάλα παράθυρα (OLS, LSS, LPS) καλεί τη βοηθητική συνάρτηση μία φορά. Για τα μικρά παράθυρα (ESH), απομονώνει καθένα από τα 8 subframes και καλεί τη βοηθητική συνάρτηση επαναληπτικά.

2. **Απομόνωση Μαθηματικών Συναρτήσεων:** Οι μαθηματικοί τύποι κβαντισμού και αποκβαντισμού `_quantize`, `_dequantize` υλοποιήθηκαν ως μικρές ανεξάρτητες συναρτήσεις.

3. **Κύριος Μαθηματικός Υπολογισμός:** Η βοηθητική συνάρτηση `_quantize_subframe` εκτελεί το "δύσκολο" έργο. Αρχικά, υπολογίζει την ενέργεια $P[b]$ και το επιτρεπόμενο κατώφλι σφάλματος $T[b]$ για κάθε μπάντα. Έπειτα, υπολογίζει την αρχική εκτίμηση του `alpha` για όλες τις μπάντες και εισέρχεται στον

επαναληπτικό βρόχο εύρεσης βέλτιστης τιμής του. Για κάθε μπάντα, κβαντίζει και αποκβαντίζει τους συντελεστές χρησιμοποιώντας τις `_quantize` και `_dequantize`, και μετράει το νέο σφάλμα `Pe_new`. Αν το σφάλμα είναι μικρότερο από το $T[b]$, το `alpha` αυξάνεται προσέχοντας να μην παραβιαστεί ο κανόνας μέγιστης διαφοράς (MAX_SCALEFACTOR = 60). Αν το σφάλμα υπερβεί το κατώφλι, η διαδικασία αναιρείται στο προηγούμενο ασφαλές βήμα.

Listing 21: Ο επαναληπτικός βρόχος ελέγχου σφάλματος

```
while True:
    # Quantize and dequantize current band
    S_new = _quantize(X[w_low:w_high + 1], alpha[b])
    X_hat_new = _dequantize(S_new, alpha[b])

    # Calculate new quantization error
    Pe_new = np.sum((X[w_low:w_high + 1] - X_hat_new) ** 2)
    all_zeros = np.all(S_new == 0)

    if Pe_new < T[b]:
        if all_zeros:
            break
        else:
            S_prev = S_new.copy()
            alpha[b] += 1
            # ... (Check MAX_SCALEFACTOR constraint) ...
    else: # Error exceeded threshold, revert to previous state
        if S_prev is not None:
            alpha[b] -= 1
            S_new = S_prev
        break
```

4. **Διαφορική Κωδικοποίηση:** Στο τέλος της `_quantize_subframe`, οι τελικές τιμές του `alpha` μετατρέπονται σε διαφορική μορφή. Η πρώτη τιμή αποθηκεύεται ως *Global Gain* (G) και ο πίνακας `sfc` αποθηκεύει μόνο τις διαφορές μεταξύ γειτονικών μπαντών.

Listing 22: Διαφορική κωδικοποίηση συντελεστών

```
sfc[0] = alpha[0]
for b in range(1, num_bands):
    sfc[b] = alpha[b] - alpha[b-1] # Differential encoding
```

5. **Αντίστροφη Διαδικασία:** Κατά την αποκωδικοποίηση (`i_aac_quantizer`), ακολουθείται η ακριβώς αντίστροφη πορεία. Η βοηθητική `_dequantize_subframe` ανακατασκευάζει πλήρως τον πίνακα των `alpha` αθροίζοντας σειριακά το G και τις τιμές του `sfc` και στη συνέχεια καλεί την `_dequantize` για να ανακτήσει τις τελικές προσεγγίσεις των συντελεστών MDCT (`X_hat`).

Η υλοποίηση της διαδικασίας κβαντισμού αναπτύσσεται στο αρχείο `aac_quantizer.py`.

4.3 Κωδικοποίηση Huffman

Η Κωδικοποίηση Huffman αποτελεί το τελικό στάδιο του κωδικοποιητή, αναλαμβάνοντας την εντροπική συμπίεση των δεδομένων. Η βαθμίδα αυτή παραλαμβάνει τα σύμβολα $S(k)$ των κβαντισμένων συντελεστών MDCT, καθώς και τις διαφορές $sfc(b)$ των κβαντισμένων scalefactors κάθε μπάντας. Σκοπός της είναι να εκχωρήσει μικρότερου μήκους δυαδικές λέξεις στα σύμβολα που εμφανίζονται συχνότερα, μειώνοντας το τελικό μέγεθος του αρχείου χωρίς να εισάγει επιπλέον σφάλμα.
Η υλοποίηση της βαθμίδας αυτής παρέχεται έτοιμη μέσω του βοηθητικού αρχείου `huff_utils.py` και των αντίστοιχων Look-Up Tables (LUTs) του αρχείου `huffCodebooks.mat`, για το οποίο έγινε μια μεταβολή ανάγνωσης του path του στο `huff_utils.py`. Οι δύο βασικές συναρτήσεις που ενσωματώνονται στο 3ο επίπεδο του συστήματος είναι οι εξής:

- `huff_sec, huff_codebook = encode_huff(coeff_sec, huff_LUT_list, force_codebook)`
  Αναλαμβάνει την κωδικοποίηση εντροπίας ενός συνόλου συντελεστών (`S`) ή scalefactors (`sfc`). Επιστρέφει την ακολουθία από '1' και '0' (`huff_sec`) και το index του codebook που τελικά επιλέχθηκε (`huff_codebook`). Βάσει προδιαγραφών, για την κωδικοποίηση των scale factors επιβάλλεται η χρήση του 11ου λεξικού θέτοντας `force_codebook=11`, ενώ για τους συντελεστές MDCT το όρισμα αυτό παραλείπεται. Ωστόσο το όρισμα εν τέλει παραλήφθηκε και στις δύο περιπτώσεις, καθώς η χρήση του δεν επέτρεπεται την διεκπαιρέωση της κωδικοποίησης (ίσως λόγω πιθανού σφάλματος στην `encode_huff`).

- `dec_coeffs = decode_huff(huff_sec, huff_codebook, huff_LUT)`
  Εκτελεί την αντίστροφη διαδικασία (αποκωδικοποίηση) των δυαδικών ακολουθιών κατά την πλευρά του αποκωδικοποιητή. Παραλαμβάνει το bitstream και το codebook που είχε χρησιμοποιηθεί κατά την κωδικοποίηση και επιστρέφει ακέραιους τους αρχικούς κβαντισμένους συντελεστές.

4.4 Encoder / Decoder 3

Το 3ο επίπεδο αποτελεί την πλήρη και τελική μορφή του απλοποιημένου κωδικοποιητή AAC της παρούσας εργασίας. Σε αυτό το στάδιο, το ηχητικό σήμα περνάει από ολόκληρη την αλυσίδα επεξεργασίας: κατατμείται σε frames, αποφασίζεται ο τύπος παραθύρου (SSC), μετασχηματίζεται στο πεδίο της συχνότητας (Filterbank), διαμορφώνεται ως προς το χρονικό σφάλμα (TNS) και, τέλος, υφίσταται απωλεστική συμπίεση. Η συμπίεση αυτή καθοδηγείται από το Ψυχοακουστικό Μοντέλο (Psychoacoustic Model), υλοποιείται από τον Κβαντιστή (Quantizer) και βελτιστοποιείται εντροπικά από την Κωδικοποίηση Huffman.
Το τελικό αποτέλεσμα του κωδικοποιητή είναι η δομή δεδομένων `aac_seq_3`. Πρόκειται για μια λίστα που περιέχει την απολύτως απαραίτητη πληροφορία για την ανακατασκευή του σήματος, προσομοιώνοντας το πραγματικό bitstream του προτύπου. Για κάθε κανάλι αποθηκεύονται: τα Huffman bitstreams για τους συντελεστές και τα scalefactors (`stream` και `sfc`), τα ευρετήρια των `codebooks`, το global gain (`G`) και οι συντελεστές `tns_coeffs`. Επίσης, αποθηκεύεται και το κατώφλι `T` καθαρά για λόγους οπτικοποίησης και μελέτης.

Λόγω της αναγκαστικής παράληψης του ορίσματος `force_codebook = 11` κατά την κωδικοποίηση των scale factors, στη δομή `aac_seq_3` έγινε προσθήκη του `sfc_codebook`, απαραίτητο για την μετέπειτα αποκωδικοποίηση. Η δομή αυτή εξάγεται στο δίσκο σε μορφή `.mat`. Αντίστροφα, ο αποκωδικοποιητής διαβάζει τη δομή, αποκωδικοποιεί τα bitstreams μέσω των inverse Huffman συναρτήσεων και εφαρμόζει διαδοχικά τις αντίστροφες βαθμίδες (`i_aac_quantizer`, `i_tns`, `i_filter_bank`) για να ανακατασκευάσει το σήμα στο πεδίο του χρόνου μέσω overlap-add.

Υλοποίηση και Δομή Κώδικα:
Η ενορχήστρωση της παραπάνω διαδικασίας υλοποιείται μέσω των βασικών συναρτήσεων `aac_coder_3(filename_in, filename_aac_coded)`, `i_aac_coder_3(aac_seq_3, filename_out)`, `demo_aac_3(filename_in, filename_out, filename_aac_coded)` και της βοηθητικής `_compute_masking_thresholds` που υπολογίζει τα κατώφλια T.

1. **Ολοκλήρωση Κωδικοποιητή:** Για κάθε frame και για κάθε κανάλι ξεχωριστά, καλούνται διαδοχικά οι βαθμίδες `psycho`, `aac_quantizer` και, τελικά, η `encode_huff` για την εντροπική συμπίεση.

   Listing 23: Η τελική αλυσίδα συμπίεσης στον κωδικοποιητή

   ```
   for ch in range(2):
   # Psychoacoustic Model
   SMR = psycho(frame_T[:, ch], frame_type, frame_T_prev_1[ch],
       frame_T_prev_2[ch])

   # Quantization
   S, sfc, G = aac_quantizer(frame_F_tns, frame_type, SMR)

   # Huffman encoding for MDCT coefficients
   stream, codebook = encode_huff(S, huff_LUT_list)

   # Huffman encoding for scale factors
   ...
   else:
       # Long frames: Single scale factor vector
       sfc_encoded, sfc_codebook = encode_huff(sfc[1:], huff_LUT_list)
   ```

   Αφού επεξεργαστούν όλα τα frames, η δομή `aac_seq_3` αποθηκεύεται σε αρχείο χρησιμοποιώντας τη συνάρτηση `scipy.io.savemat`.

2. **Αποκωδικοποιητής:** Στον αποκωδικοποιητή, το πρώτο βήμα για κάθε frame είναι η μετατροπή των bitstreams ξανά σε ακέραιους αριθμούς, καλώντας την `decode_huff`. Στη συνέχεια, τα δεδομένα τροφοδοτούνται στον `i_aac_quantizer` για την ανακατασκευή των συντελεστών MDCT, και έπεται η κλήση των `i_tns` και `i_filter_bank`.

3. **Αξιολόγηση Συστήματος** Η συνάρτηση `demo_aac_3` εκτελεί ολόκληρο το pipeline και υπολογίζει τις τρεις βασικές μετρικές απόδοσης του συστήματος:

- **SNR (Signal-to-Noise Ratio):** Υπολογίζεται από τον λόγο της ενέργειας του αρχικού σήματος προς την ενέργεια του σφάλματος.
- **Bitrate (bits/sec):** Για τον υπολογισμό του παραγόμενου bitrate, αθροίζεται το μήκος των strings (bits) των `stream` και `sfc` όλων των frames, μαζί με μια εκτίμηση του overhead (bits που απαιτούνται για την αποθήκευση των `G`, `frame_type`, `tns_coeffs` κ.λπ.). Το συνολικό άθροισμα διαιρείται με τη χρονική διάρκεια του ηχητικού αρχείου σε δευτερόλεπτα
- **Λόγος Συμπίεσης (Compression Ratio):** Συγκρίνεται το παραγόμενο bitrate με το αρχικό bitrate του ασυμπίεστου σήματος. Για ένα στερεοφωνικό αρχείο wav στα 48kHz και 16 bits ανά δείγμα, το αρχικό bitrate είναι $48000 \times 2 \times 16 = 1.536$ Mbps

Listing 24: Υπολογισμός Bitrate και Compression Ratio

```python
# Calculate duration of original audio
duration = len(original_audio) / fs

# Original bitrate: 48000 Hz * 2 channels * bit_depth bits/sample
original_bitrate = fs * 2 * bit_depth

# Calculate bitrate and compression ratio
bitrate = total_bits / duration
compression = original_bitrate / bitrate

# Calculate SNR
signal_power = np.sum(original_audio ** 2)
noise_power = np.sum((original_audio - decoded_audio) ** 2)

if noise_power == 0:
    SNR = float('inf')
else:
    SNR = 10 * np.log10(signal_power / noise_power)
```

Συμπερασματικά, το 3ο επίπεδο επιδεικνύει την ικανότητα του προτύπου AAC να επιτυγχάνει σημαντικούς λόγους συμπίεσης, εκμεταλλευόμενο τις ατέλειες της ανθρώπινης ακοής, διατηρώντας ταυτόχρονα υψηλή αντιληπτή ποιότητα (υψηλό SNR). Η παραπάνω υλοποίηση βρίσκεται στο αρχείο `aac_codec_3.py` και στο επιπλέον test script `aac_test_3.py` που υλοποιήθηκε για την εύκολη εκτέλεση της επίδειξης:

Listing 25: Αποτελέσματα Επιπέδου 3

```
...
Total frames: 277
Frame types: OLS=247, LSS=10, ESH=10, LPS=10

Compression Statistics and Audio Quality:
------------------------------------------------------------
SNR: 10.07 dB
Compressed Bitrate: 197442 bits/sec (197.4 kbps)
```

```
Compression ratio: 7.78:1

Quality: Poor
```

Από τα παραπάνω αποτελέσματα επιβεβαιώνεται η θεμελιώδης αρχή της απωλεστικής συμπίεσης ήχου (lossy audio compression). Το σύστημα πέτυχε σημαντικότατο λόγο συμπίεσης (7.78:1), μειώνοντας ραγδαία τον ρυθμό μετάδοσης από τα 1536 kbps του ασυμπίεστου αρχείου στα 197.4 kbps. Αυτή η δραστική μείωση της πληροφορίας προκάλεσε αναμενόμενα την εισαγωγή αξιοσημείωτου θορύβου κβαντισμού, γεγονός που αποτυπώνεται στο χαμηλό SNR (10.07 dB).

Ωστόσο, βάσει της θεωρίας των ψυχοακουστικών κωδικοποιητών, το σφάλμα αυτό δεν κατανέμεται τυχαία, αλλά "κρύβεται" κάτω από τα όρια της ανθρώπινης ακοής. Για την οπτική επαλήθευση αυτής της διαδικασίας, παρατίθεται το Σχήμα 1, το οποίο απεικονίζει τα υπολογισμένα κατώφλια ακουστότητας $T(b)$ σε dB, ανά συχνοτική μπάντα (scalefactor band) για ένα τυχαίο Long Frame (69 μπάντες):

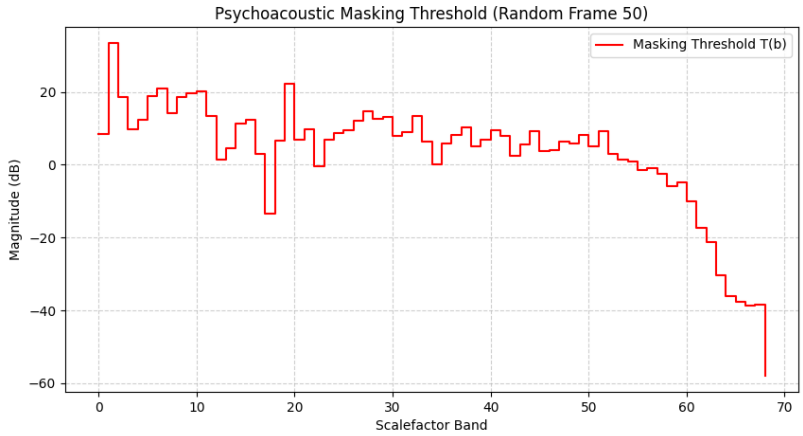

Figure 1: Ψυχοακουστικό κατώφλι μάσκας $T(b)$ ανά Scalefactor Band για τυχαίο frame.

Όπως παρατηρούμε στο γράφημα, η τιμή του κατωφλίου μεταβάλλεται δυναμικά (ως βηματική συνάρτηση) ανάλογα με το ενεργειακό περιεχόμενο της κάθε μπάντας. Στις χαμηλές και μεσαίες μπάντες, όπου η ενέργεια του σήματος είναι συνήθως μεγαλύτερη, το σύστημα "χαλαρώνει" τον κβαντιστή, επιτρέποντας την εισαγωγή μεγαλύτερου σφάλματος κβαντισμού. Αντίθετα, στα σημεία όπου το κατώφλι πέφτει κατακόρυφα (όπως στις πολύ υψηλές συχνότητες), ο κβαντιστής αναγκάζεται να χρησιμοποιήσει περισσότερα bits για να διατηρήσει το σφάλμα μη αντιληπτό ή μηδενίζει τους συντελεστές αν απουσιάζει τελείως το σήμα. Αυτή η δυναμική προσαρμογή είναι που επιτρέπει στον κωδικοποιητή AAC να πετυχαίνει τη βέλτιστη κατανομή της πληροφορίας και τόσο υψηλούς λόγους συμπίεσης.